

Global Institute of Technology

UNIT 3

SEARCHING AND SORTING ALGORITHMS

INTRODUCTION TO SEARCHING ALGORITHMS

Searching is an operation or a technique that helps find the place of a given element or value in the list. Any search is said to be successful or unsuccessful depending upon whether the element that is being searched is found or not. Some of the standard searching techniques that are being followed in data structure are listed below:

1. Linear Search
2. Binary Search

LINEAR SEARCH

Linear search is a very basic and simple search algorithm. In Linear search, we search an element or value in a given array by traversing the array from the starting, till the desired element or value is found.

It compares the element to be searched with all the elements present in the array and when the element is **matched** successfully, it returns the index of the element in the array, else it returns -1.

Linear Search is applied on unsorted or unordered lists, when there are fewer elements in a list. For Example,

Linear Search

10 14 19 26 27 31 33 35 42 44
=33

Algorithm

Linear Search (Array A, Value x)

Step 1: Set i to 1

Step 2: if $i > n$ then go to step 7

Step 3: if $A[i] = x$ then go to step 6

Step 4: Set i to $i + 1$

Step 5: Go to Step 2

Step 6: Print Element x Found at index i and go to step 8

Step 7: Print element not found

Step 8: Exit Pseudocode

procedure linear_search (list, value)

for each item in the list if match item == value

 return the item's location

end if

end for

end procedure

BINARY SEARCH

A binary search is an advanced type of search algorithm that finds and fetches data from a sorted list of items. Its core working principle involves dividing the data in the list to half until the required value is located and displayed to the user in the search result. Binary search is commonly known as a **half-interval search** or a **logarithmic search**.

Binary Search is used with sorted array or list. In binary search, we follow the following steps:

1. We start by comparing the element to be searched with the element in the middle of the list/array.
2. If we get a match, we return the index of the middle element.
3. If we do not get a match, we check whether the element to be searched is less or greater than in value than the middle element.

4. If the element/number to be searched is greater in value than the middle number, then we pick the elements on the right side of the middle element (as the list/array is sorted, hence on the right, we will have all the numbers greater than the middle number), and start again from the step 1.

5. If the element/number to be searched is lesser in value than the middle number, then we pick the elements on the left side of the middle element, and start again from the step 1.

Binary Search is useful when there are large numbers of elements in an array and they are sorted. So a necessary condition for Binary search to work is that the list/array should be sorted.

Features of Binary Search

1. It is great to search through large sorted arrays.
2. It has a time complexity of $O(\log n)$ which is a very good time complexity. It has a simple implementation.

Binary search is a fast search algorithm with run-time complexity of $O(\log n)$. This search algorithm works on the principle of divide and conquers. For this algorithm to work properly, the data collection should be in the sorted form. Binary search looks for a particular item by comparing the middle most item of the collection. If a match occurs, then the index of item is returned. If the middle item is greater than the item, then the item is searched in the sub-array to the left of the middle item. Otherwise, the item is searched for in the sub-array to the right of the middle item. This process continues on the sub- array as well until the size of the sub array reduces to zero.

Binary Search										
	0	1	2	3	4	5	6	7	8	9
Search 23	2	5	8	12	16	23	38	56	72	91
	L=0	1	2	3	M=4	5	6	7	8	H=9
23 > 16 take 2 nd half	2	5	8	12	16	23	38	56	72	91
	0	1	2	3	4	L=5	6	M=7	8	H=9
23 < 56 take 1 st half	2	5	8	12	16	23	38	56	72	91
	0	1	2	3	4	L=5, M=5	H=6	7	8	9
Found 23, Return 5	2	5	8	12	16	23	38	56	72	91

]

Sorting

Types of Sorting Algorithms

Bubble Sort

Bubble sort is one of the simplest sorting algorithms that continually steps through the index, compares adjacent components and swaps them if they are placed in the incorrect order.

Input: arr[] = {5, 1, 4, 2, 8}

First Pass:

- Bubble sort starts with very first two elements, comparing them to check which one is greater.
- (5 1 4 2 8) -> (1 5 4 2 8), Here, algorithm compares the first two elements, and swaps since $5 > 1$.
- (1 5 4 2 8) -> (1 4 5 2 8), Swap since $5 > 4$
- (1 4 5 2 8) -> (1 4 2 5 8), Swap since $5 > 2$
- (1 4 2 5 8) -> (1 4 2 5 8), Now, since these elements are already in order ($8 > 5$), algorithm does not swap them.

Second Pass:

Now, during second iteration it should look like this:

- (**1 4** 2 5 8) → (**1 4** 2 5 8)
- (**1 4** 2 5 8) → (**1 2 4** 5 8), Swap since $4 > 2$
- (**1 2 4** 5 8) → (**1 2 4** 5 8)
- (**1 2 4 5** 8) → (**1 2 4 5** 8)

Third Pass:

- Now, the array is already sorted, but our algorithm does not know if it is completed.
- The algorithm needs one **whole** pass without **any** swap to know it is sorted.
- (**1 2 4 5** 8) → (**1 2 4 5** 8)
- (**1 2 4 5** 8) → (**1 2 4 5** 8)
- (**1 2 4 5** 8) → (**1 2 4 5** 8)
- (**1 2 4 5** 8) → (**1 2 4 5** 8)

Selection Sort

Selection sort finds the smallest component in the list and puts it in the first position on the list, then it searches for the second last element in the array and puts it in the second position. This procedure continues until all the components are transferred to their accurate placement.

Lets consider the following array as an example: `arr[] = {64, 25, 12, 22, 11}`

First pass:

- For the first position in the sorted array, the whole array is traversed from index 0 to 4 sequentially. The first position where **64** is stored presently, after traversing whole array it is clear that **11** is the lowest value.

64 25 12 22 11

- Thus, replace 64 with 11. After one iteration **11**, which happens to be the least value in the array, tends to appear in the first position of the sorted list.

11 25 12 22 64

Second Pass:

- For the second position, where 25 is present, again traverse the rest of the array in a sequential manner.

11 **25** 12 22 64

- After traversing, we found that **12** is the second lowest value in the array and it should appear at the second place in the array, thus swap these values.

11 **12** 25 22 64

Third Pass:

- Now, for third place, where 25 is present again traverse the rest of the array and find the third least value present in the array.

11 12 **25** 22 64

- While traversing, **22** came out to be the third least value and it should appear at the third place in the array, thus swap **22** with element present at third position.

11 12 **22** 25 64

Fourth pass:

- Similarly, for fourth position traverse the rest of the array and find the fourth least element in the array
- As **25** is the 4th lowest value hence, it will place at the fourth position.

11 12 22 **25** 64

Fifth Pass:

- At last the largest value present in the array automatically get placed at the last position in the array
- The resulted array is the sorted array.

11 12 22 25 64

Insertion Sort

Insertion sorting is an uncomplicated sorting technique that inserts each component of the array to its appropriate position.

Characteristics of Insertion Sort:

- This algorithm is one of the simplest algorithm with simple implementation
- Basically, Insertion sort is efficient for small data values
- Insertion sort is adaptive in nature, i.e. it is appropriate for data sets which are already partially sorted.

Consider an example: `arr[]: {12, 11, 13, 5, 6}`

12 11 13 5 6

First Pass:

- Initially, the first two elements of the array are compared in insertion sort.

12 11 13 5 6

- Here, 12 is greater than 11 hence they are not in the ascending order and 12 is not at its correct position. Thus, swap 11 and 12.
- So, for now 11 is stored in a sorted sub-array.

11 12 13 5 6

Second Pass:

- Now, move to the next two elements and compare them

11 **12 13** 5 6

- Here, 13 is greater than 12, thus both elements seems to be in ascending order, hence, no swapping will occur. 12 also stored in a sorted sub-array along with 11

Third Pass:

- Now, two elements are present in the sorted sub-array which are **11 and 12**
- Moving forward to the next two elements which are 13 and 5

11 12 **13 5** 6

- Both 5 and 13 are not present at their correct place so swap them

11 12 5 **13** 6

- After swapping, elements 12 and 5 are not sorted, thus swap again

11 5 **12** 13 6

- Here, again 11 and 5 are not sorted, hence swap again

5 11 12 13 6

- here, it is at its correct position

Fourth Pass:

- Now, the elements which are present in the sorted sub-array are **5, 11 and 12**
- Moving to the next two elements 13 and 6

5 11 12 **13** **6**

- Clearly, they are not sorted, thus perform swap between both

5 11 12 **6** **13**

- Now, 6 is smaller than 12, hence, swap again

5 11 **6** **12** 13

- Here, also swapping makes 11 and 6 unsorted hence, swap again

5 **6** **11** 12 13

- Finally, the array is completely sorted.

Merge Sort

Merge sort prefers the divide and conquer strategy to organize the elements. It is one of the most efficient sorting algorithms. It breaks the provided array into two parts, then sorts them and merges them.

Merge Sort Working Process:

Think of it as a recursive algorithm continuously splits the array in half until it cannot be further divided. This means that if the array becomes empty or has only one element left, the dividing will stop, i.e. it is the base case to stop the recursion. If the array has multiple elements, split the array into halves and recursively invoke the merge sort on each of the halves. Finally, when both halves are sorted, the merge operation is applied. Merge operation is the process of taking two smaller sorted arrays and combining them to eventually make a larger one.

Algorithm for Merge Sort: MergeSort(array, first, last)

Step 1: Find the middle index of the array.

Middle = $1 + (\text{last} - \text{first})/2$

Step 2: Divide the array from the middle.

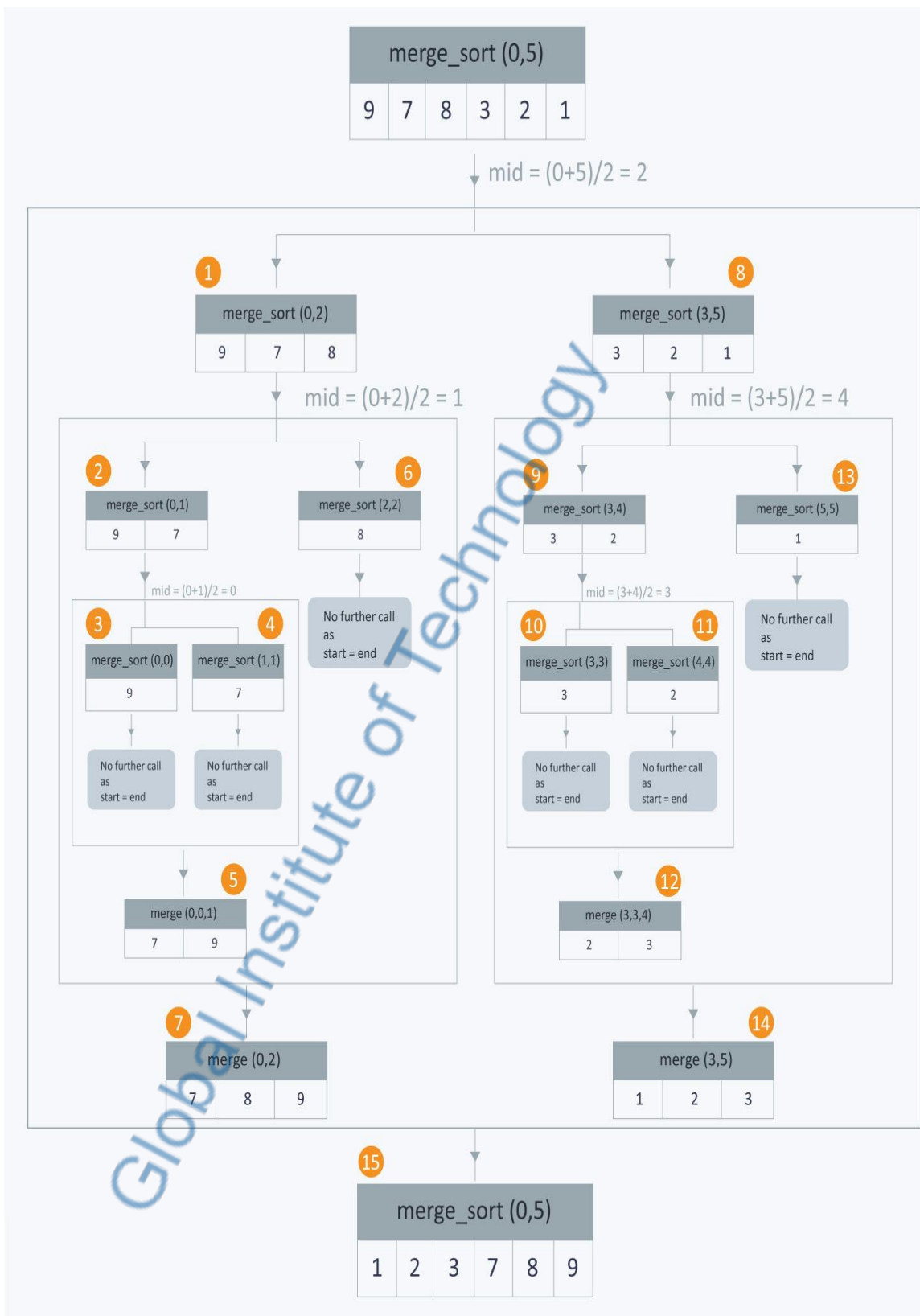
Step 3: Call merge sort for the first half of the array

MergeSort(array, first, middle)

Step 4: Call merge sort for the second half of the array.

MergeSort(array, middle+1, last)

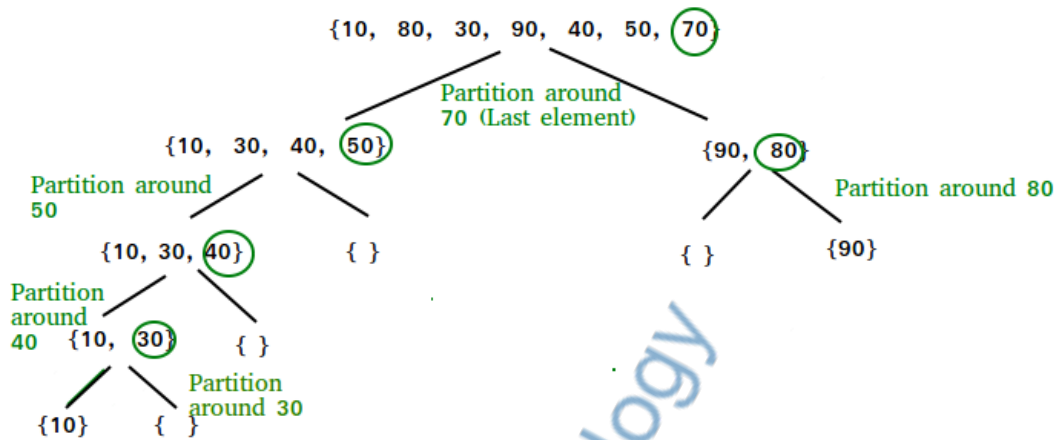
Step 5: Merge the two sorted halves into a single sorted array.



Quicksort

Quick sort is based on the divide-and-conquer approach based on the idea of choosing one element as a pivot element and partitioning the array around it such that: Left side of pivot contains all the elements that are less than the pivot element Right side contains all elements greater than the pivot

It reduces the space complexity and removes the use of the auxiliary array that is used in merge sort. Selecting a random pivot in an array results in an improved time complexity in most of the cases.



Counting Sort

Counting sort is a sorting algorithm that sorts an array's elements by estimating the appearance of unique elements in the array.

Characteristics of counting sort:

- Counting sort makes assumptions about the data, for example, it assumes that values are going to be in the range of 0 to 10 or 10 – 99 etc, Some other assumptions counting sort makes are input data will be all real numbers.
- Like other algorithms this sorting algorithm is not a comparison-based algorithm, it hashes the value in a temporary count array and uses them for sorting.
- It uses a temporary array making it a non In Place algorithm.

Example:

2	9	7	4	1	8	4
---	---	---	---	---	---	---

- Find the maximum element from the given array. Let max be the maximum element.

max						
9	2	7	4	1	8	4

- Now, initialize array of length max + 1 having all 0 elements. This array will be used to store the count of the elements in the given array.

0	1	2	3	4	5	6	7	8	9
0	0	0	0	0	0	0	0	0	0

Count array

- Now, we have to store the count of each array element at their corresponding index in the count array.

The count of an element will be stored as - Suppose array element '4' is appeared two times, so the count of element 4 is 2. Hence, 2 is stored at the 4th position of the count array. If any element is not present in the array, place 0, i.e. suppose element '3' is not present in the array, so, 0 will be stored at 3rd position.

Given array	2	9	7	4	1	8	4			
	0	1	2	3	4	5	6	7	8	9
Count array	0	1	1	0	2	0	0	1	1	1
Count of each stored element										

Now, store the cumulative sum of count array elements. It will help to place the elements at the correct index of the sorted array.

0	1	2	3	4	5	6	7	8	9
0	1	2	0	2	0	0	1	1	1
1+1=2									
0	1	2	3	4	5	6	7	8	9
0	1	2	2	2	0	0	1	1	1
2+0=2									

Similarly, the cumulative count of the count array is -

0	1	2	3	4	5	6	7	8	9
0	1	2	2	4	4	4	5	6	7
Cumulative count									

4. Now, find the index of each element of the original array

For element 9

Original array	2	9	7	4	1	8	4			
	0	1	2	3	4	5	6	7	8	9
Count array	0	1	1	2	4	4	4	5	6	7
										7 - 1 = 6
Output (sorted array)	0	1	2	3	4	5	6			
		2					9			

After placing element at its place, decrease its count by one. Before placing element 2, its count was 2, but after placing it at its correct position, the new count for element 2 is 1.

For element 2

Original array	<table><tr><td>2</td><td>9</td><td>7</td><td>4</td><td>1</td><td>8</td><td>4</td></tr></table>								2	9	7	4	1	8	4					
2	9	7	4	1	8	4														
	<table><tr><td>0</td><td>1</td><td>2</td><td>3</td><td>4</td><td>5</td><td>6</td><td>7</td><td>8</td><td>9</td></tr></table>								0	1	2	3	4	5	6	7	8	9		
0	1	2	3	4	5	6	7	8	9											
Count array	<table><tr><td>0</td><td>1</td><td>2</td><td>2</td><td>4</td><td>4</td><td>4</td><td>5</td><td>6</td><td>7</td></tr></table>										0	1	2	2	4	4	4	5	6	7
0	1	2	2	4	4	4	5	6	7											
	<table><tr><td></td><td></td><td>2-1=1</td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr></table>												2-1=1							
		2-1=1																		
Output (sorted array)	<table><tr><td></td><td>2</td><td></td><td></td><td></td><td></td><td></td></tr></table>								2											
	2																			

Similarly, after sorting, the array elements are -

Output (sorted array)	1	2	4	4	7	8	9			
-----------------------	---	---	---	---	---	---	---	--	--	--

Radix Sort

In the Radix sort algorithm, the sorting is performed based on a dictionary or alphabetical order. It is the linear sorting algorithm that we mostly preferred for integers.

Algorithm

1. radixSort(arr)
2. max = largest element in the given array
3. d = number of digits in the largest element (or, max)
4. Now, create d buckets of size 0 - 9
5. **for** i -> 0 to d
6. sort the array elements using counting sort (or any stable sort) according to the digits at the ith place
7. the ith place

Example

181	289	390	121	145	736	514	212
-----	-----	-----	-----	-----	-----	-----	-----

Pass 1:

In the first pass, the list is sorted on the basis of the digits at 0's place.

181		390	0
289		181	1
390		121	1
121		212	2
145			3
736		514	4
514		145	5
212		736	6
			7
			8
		289	9

After the first pass, the array elements are -

390	181	121	212	514	145	736	289
-----	-----	-----	-----	-----	-----	-----	-----

Pass 2:

In this pass, the list is sorted on the basis of the next significant digits (i.e., digits at 10th place).

3	9	0	
1	8	1	
1	2	1	
2	1	2	
5	1	4	
1	4	5	
7	3	6	
2	8	9	

	0
212	1
514	2
121	3
736	4
145	5
	6
	7
181	8
289	9

After the second pass, the array elements are -

212	514	121	736	145	181	289	390
-----	-----	-----	-----	-----	-----	-----	-----

Pass 3:

In this pass, the list is sorted on the basis of the next significant digits (i.e., digits at 100th place).

2	12	
5	14	
1	21	
7	36	
1	45	
1	81	
2	89	
3	90	

	0
121	1
145	2
181	3
212	4
289	5
390	6
	7
514	8
	9

After the third pass, the array elements are -

121	145	181	212	289	390	514	736
-----	-----	-----	-----	-----	-----	-----	-----

Now, the array is sorted in ascending order.

Bucket Sort

In the bucket sort algorithm, the buckets are sorted separately by utilizing distinct sorting algorithms.

Bucket sort is a sorting algorithm that separate the elements into multiple groups said to be buckets. Elements in bucket sort are first uniformly divided into groups called buckets, and then they are sorted by any other sorting algorithm. After that, elements are gathered in a sorted manner.

The basic procedure of performing the bucket sort is given as follows -

- First, partition the range into a fixed number of buckets.

- Then, toss every element into its appropriate bucket.
- After that, sort each bucket individually by applying a sorting algorithm.
- And at last, concatenate all the sorted buckets.

Advantages

- Bucket sort reduces the no. of comparisons.
- It is asymptotically fast because of the uniform distribution of elements.

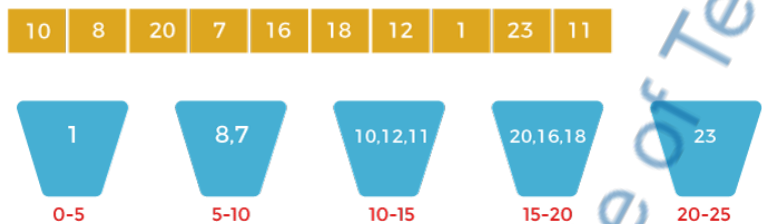
The **limitations** of bucket sort are -

- It may or may not be a stable sorting algorithm.
- It is not useful if we have a large array because it increases the cost.
- It is not an in-place sorting algorithm, because some extra space is required to sort the buckets.

The best and average-case complexity of bucket sort is **$O(n + k)$** , and the worst-case complexity of bucket sort is **$O(n^2)$** , where n is the number of items.

Bucket sort is commonly used -

- With floating-point values.
- When input is distributed uniformly over a range.



Now, **sort** each bucket individually. The elements of each bucket can be sorted by using any of the stable sorting algorithms.



At last, gather the sorted elements from each bucket in order



Heap Sort

It is a popular comparison-based sorting approach based on Binary Heap data structure. It is very much similar to the selection sort algorithm where we first encounter the lowest element and put the lowest element at the front.

What Is Heap Sort?

Binary Heap

Heap is a tree-based data structure in which all the tree nodes are in a particular order, such that the tree satisfies the heap properties (that is, there is a specific parent-child relationship that is followed throughout the tree).

A heap data structure where the tree is a complete binary tree is referred to as a binary heap.

A **complete binary tree** is a binary tree in which all levels except the bottom-most level are completely filled, and all nodes in the bottom-most level are as far left as possible. (The last level may or may not be completely filled.)

A **full binary tree** is a binary tree where every node has 0 or 2 children.

Properties of a Binary Heap

1. They are complete binary trees: This means all levels are totally filled (except maybe the last level), and the nodes in the last level are as left as possible. This property makes arrays a suitable data structure for storing binary heaps.

We can easily calculate indices of a node's children. So, for parent index i , the left child will be found at index $2*i+1$, and the right child will be found at index $2*i+2$ (for indices that start with 0). Similarly, for a child at index i , its parent can be found at index $i/2$.

2. Heaps are typically of two types — max heap and min heap: In a max heap, the value of a node is always greater than or equal to the value of each of its children. Conversely, in a min heap, the value of a parent is always $\leq$ the value of each of its children.

3. In a max heap, the element at the root will always be the maximum. In a min heap, the element at the root will always be the minimum. Heap sort algorithm takes advantage of this property to sort an array using heaps.

Algorithm

1. HeapSort(arr)
2. BuildMaxHeap(arr)
3. for $i = \text{length}(\text{arr})$ to 2
4. swap arr[1] with arr[i]
5. heap_size[arr] = heap_size[arr] - 1
6. MaxHeapify(arr,1)
7. End

BuildMaxHeap(arr)

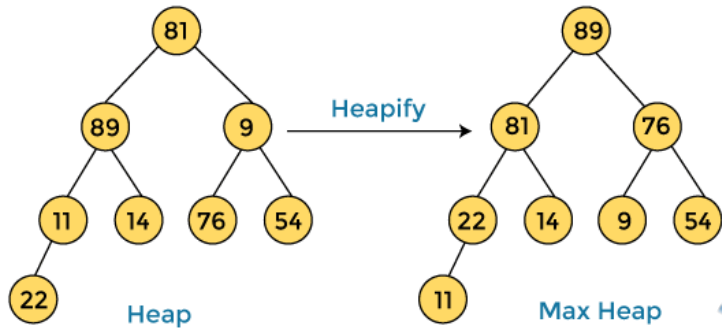
1. BuildMaxHeap(arr)
2. heap_size(arr) = length(arr)
3. for $i = \text{length}(\text{arr})/2$ to 1
4. MaxHeapify(arr,i)
5. End

MaxHeapify(arr,i)

1. MaxHeapify(arr,i)
 2. $L = \text{left}(i)$
 3. $R = \text{right}(i)$
 4. if $L \neq \text{heap_size}[\text{arr}]$ and $\text{arr}[L] > \text{arr}[i]$
 5. largest = L
 6. else
 7. largest = i
 8. if $R \neq \text{heap_size}[\text{arr}]$ and $\text{arr}[R] > \text{arr}[\text{largest}]$
-

- 9. largest = R
- 10. if largest != i
- 11. swap arr[i] with arr[largest]
- 12. MaxHeapify(arr,largest)
- 13. End

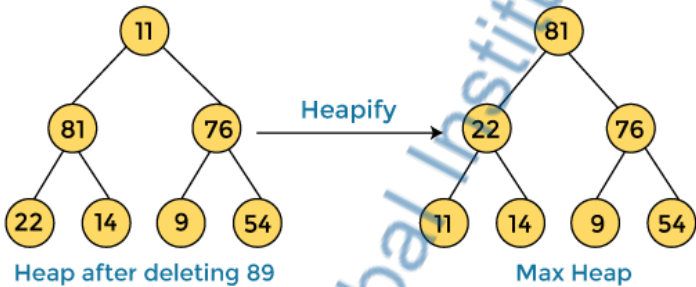
81	89	9	11	14	76	54	22
----	----	---	----	----	----	----	----



After converting the given heap into max heap, the array elements are -

89	81	76	22	14	9	54	11
----	----	----	----	----	---	----	----

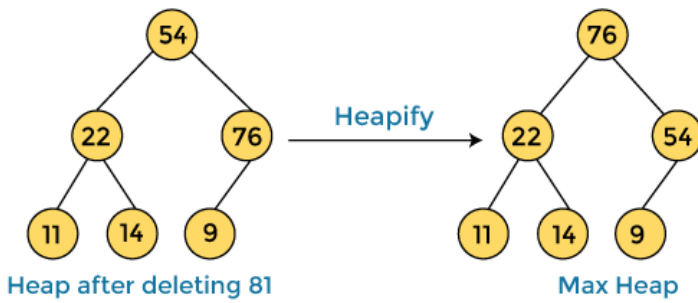
Next, we have to delete the root element (89) from the max heap. To delete this node, we have to swap it with the last node, i.e. (11). After deleting the root element, we again have to heapify it to convert it into max heap.



After swapping the array element 89 with 11, and converting the heap into max-heap, the elements of array are -

81	22	76	11	14	9	54	89
----	----	----	----	----	---	----	----

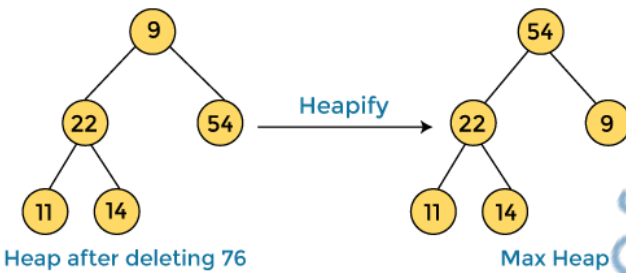
In the next step, again, we have to delete the root element (81) from the max heap. To delete this node, we have to swap it with the last node, i.e. (54). After deleting the root element, we again have to heapify it to convert it into max heap.



After swapping the array element 81 with 54 and converting the heap into max-heap, the elements of array are -

76	22	54	11	14	9	81	89
----	----	----	----	----	---	----	----

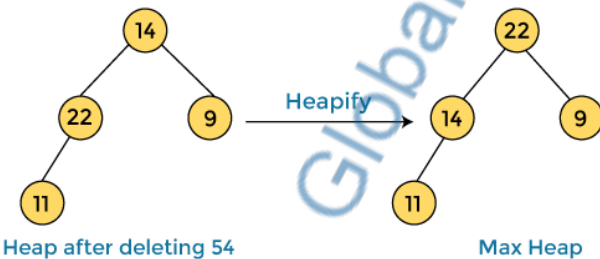
In the next step, we have to delete the root element (76) from the max heap again. To delete this node, we have to swap it with the last node, i.e. (9). After deleting the root element, we again have to heapify it to convert it into max heap.



After swapping the array element 76 with 9 and converting the heap into max-heap, the elements of array are -

54	22	9	11	14	76	81	89
----	----	---	----	----	----	----	----

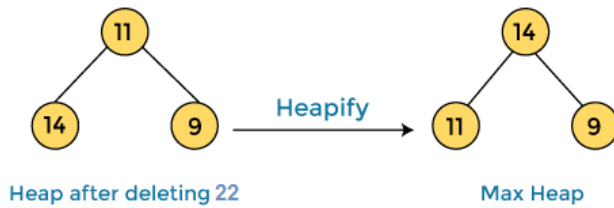
In the next step, again we have to delete the root element (54) from the max heap. To delete this node, we have to swap it with the last node, i.e. (14). After deleting the root element, we again have to heapify it to convert it into max heap.



After swapping the array element 54 with 14 and converting the heap into max-heap, the elements of array are -

22	14	9	11	54	76	81	89
----	----	---	----	----	----	----	----

In the next step, again we have to delete the root element (22) from the max heap. To delete this node, we have to swap it with the last node, i.e. (11). After deleting the root element, we again have to heapify it to convert it into max heap.



After swapping the array element 22 with 11 and converting the heap into max-heap, the elements of array are -

14	11	9	22	54	76	81	89
----	----	---	----	----	----	----	----

In the next step, again we have to delete the root element (14) from the max heap. To delete this node, we have to swap it with the last node, i.e. (9). After deleting the root element, we again have to heapify it to convert it into max heap.



After swapping the array element 14 with 9 and converting the heap into max-heap, the elements of array are -

11	9	14	22	54	76	81	89
----	---	----	----	----	----	----	----

In the next step, again we have to delete the root element (11) from the max heap. To delete this node, we have to swap it with the last node, i.e. (9). After deleting the root element, we again have to heapify it to convert it into max heap.



After swapping the array element 11 with 9, the elements of array are -

9	11	14	22	54	76	81	89
---	----	----	----	----	----	----	----

Now, heap has only one element left. After deleting it, heap will be empty.



After completion of sorting, the array elements are -

9	11	14	22	54	76	81	89
---	----	----	----	----	----	----	----

Now, the array is completely sorted.

Global Institute of Technology

Notes